

BubbleSort – Klassiker

Pseudocode (IHK-Stil):

```
Funktion bubbleSort(A: Array<Integer>)  
  n = länge(A)  
  for i = 0 to n-2  
    for j = 0 to n-i-2  
      if A[j] > A[j+1] dann  
        tausche A[j], A[j+1]  
      end if  
    end for  
  end for  
end Funktion
```

Java-Beispiel:

```
public static void bubbleSort(int[] A) {  
  int n = A.length;  
  for (int i = 0; i < n-1; i++) {  
    for (int j = 0; j < n-i-1; j++) {  
      if (A[j] > A[j+1]) {  
        int tmp = A[j];  
        A[j] = A[j+1];  
        A[j+1] = tmp;  
      }  
    }  
  }  
}
```

BubbleSort – Erklärung & Beispiel

Vergleicht Nachbarelemente mehrfach und tauscht sie. Das größte Element „blubbert“ nach rechts.

Beispielarray: [5,3,1]

Typische Fehler: falsche Schleifengrenzen, nur einmal tauschen.

Prüfungstipp: Sehr beliebt als Einstiegsaufgabe.

Schritt	Array	Bemerkung
Start	[5,3,1]	unsortiert
1	[3,5,1]	5 wandert nach rechts
2	[3,1,5]	1 nach links geschoben
3	[1,3,5]	fertig sortiert

InsertionSort – Einfügesortieren

Pseudocode (IHK-Stil):

```
Funktion insertionSort(A: Array<Integer>)  
  n = länge(A)  
  for i = 1 to n-1  
    key = A[i]  
    j = i - 1  
    while j >= 0 und A[j] > key  
      A[j+1] = A[j]  
      j = j - 1  
    end while  
    A[j+1] = key  
  end for  
end Funktion
```

Java-Beispiel:

```
public static void insertionSort(int[] A) {  
  int n = A.length;  
  for (int i = 1; i < n; i++) {  
    int key = A[i];  
    int j = i-1;  
    while (j >= 0 && A[j] > key) {  
      A[j+1] = A[j];  
      j--;  
    }  
    A[j+1] = key;  
  }  
}
```

InsertionSort – Erklärung & Beispiel

Fügt Elemente in den sortierten linken Bereich ein.

Beispielarray: [5,2,4,1]

Typische Fehler: key nicht richtig einsetzen.

Prüfungstipp: Gut für fast sortierte Arrays.

Schritt	Array	Bemerkung
Start	[5,2,4,1]	unsortiert
i=1	[2,5,4,1]	2 eingefügt
i=2	[2,4,5,1]	4 eingefügt
i=3	[1,2,4,5]	1 eingefügt → fertig

SelectionSort – Auswahl sortieren

Pseudocode (IHK-Stil):

```
Funktion selectionSort(A: Array<Integer>)  
  n = länge(A)  
  for i = 0 to n-2  
    minIndex = i  
    for j = i+1 to n-1  
      if A[j] < A[minIndex] dann  
        minIndex = j  
      end if  
    end for  
    tausche A[i], A[minIndex]  
  end for  
end Funktion
```

Java-Beispiel:

```
public static void selectionSort(int[] A) {  
  int n = A.length;  
  for (int i = 0; i < n-1; i++) {  
    int minIndex = i;  
    for (int j = i+1; j < n; j++) {  
      if (A[j] < A[minIndex]) minIndex = j;  
    }  
    int tmp = A[i];  
    A[i] = A[minIndex];  
    A[minIndex] = tmp;  
  }  
}
```

SelectionSort – Erklärung & Beispiel

Sucht jeweils das kleinste Element und tauscht es nach vorn.

Beispielarray: [64,25,12,22,11]

Typische Fehler: minIndex nicht zurücksetzen.

Prüfungstipp: Vergleich zu BubbleSort wichtig.

Schritt	Array	Bemerkung
Start	[64,25,12,22,11]	unsortiert
i=0	[11,25,12,22,64]	11 nach vorne
i=1	[11,12,25,22,64]	12 nach vorne
i=2	[11,12,22,25,64]	22 nach vorne
i=3	[11,12,22,25,64]	fertig

MergeSort – Rekursives Teilen und Mergen

Pseudocode (IHK-Stil):

```
Funktion mergeSort(A: Array<Integer>)
  if länge(A) ≤ 1 dann return A
  mitte = länge(A) / 2
  links = mergeSort(A[0..mitte-1])
  rechts = mergeSort(A[mitte..ende])
  return merge(links, rechts)
end Funktion

Funktion merge(L, R) → Array<Integer>
  i=j=k=0
  erg = neues Array[Länge(L)+Länge(R)]
  while i < Länge(L) und j < Länge(R)
    if L[i] ≤ R[j] dann
      erg[k]=L[i]; i++
    else
      erg[k]=R[j]; j++
    end if
    k++
  end while
  rest anhängen
  return erg
end Funktion
```

Java-Beispiel:

```
public static void mergeSort(int[] A) {
  if (A.length <= 1) return;
  int mid = A.length/2;
  int[] left = Arrays.copyOfRange(A,0,mid);
  int[] right = Arrays.copyOfRange(A,mid,A.length);
  mergeSort(left);
  mergeSort(right);
  merge(A, left, right);
}
```

MergeSort – Erklärung & Beispiel

Teilt das Array in Hälften, sortiert rekursiv und fügt zusammen.

Beispielarray: [38,27,43,3,9,82,10]

Komplexität: $O(n \log n)$.

Prüfungstipp: Rekursives Denken üben.

Schritt	Teilarrays	Bemerkung
Start	[38,27,43,3,9,82,10]	
Teilen	[38,27,43] [3,9,82,10]	halbieren
Weiter	[38] [27,43] / [3,9] [82,10]	rekursiv
Merge	[27,38,43] [3,9,10,82]	sortiert
Ende	[3,9,10,27,38,43,82]	fertig

QuickSort – Rekursiv mit Pivot

Pseudocode (IHK-Stil):

```
Funktion quickSort(A: Array<Integer>, low: Integer, high: Integer)
  if low < high dann
    p = partition(A, low, high)
    quickSort(A, low, p-1)
    quickSort(A, p+1, high)
  end if
end Funktion
```

```
Funktion partition(A, low, high) → Integer
  pivot = A[high]
  i = low-1
  for j = low to high-1
    if A[j] ≤ pivot dann
      i++
      tausche A[i], A[j]
    end if
  end for
  tausche A[i+1], A[high]
  return i+1
end Funktion
```

Java-Beispiel:

```
public static void quickSort(int[] A, int low, int high) {
  if (low < high) {
    int p = partition(A, low, high);
    quickSort(A, low, p-1);
    quickSort(A, p+1, high);
  }
}
```

QuickSort – Erklärung & Beispiel

Wählt ein Pivot-Element, partitioniert Array und sortiert Teilarrays rekursiv.

Beispielarray: [10,7,8,9,1,5], pivot=5

Komplexität: $O(n \log n)$ durchschnittlich, $O(n^2)$ im Worst-Case.

Prüfungstipp: Partition verstehen ist wichtig.

Schritt	Array	Bemerkung
Start	[10,7,8,9,1,5]	pivot=5
Partition	[1,5,8,9,10,7]	5 an richtige Stelle
Rekursiv	[1] [5] [8,9,10,7]	Teile
Weiter	[1,5,7,8,9,10]	fertig sortiert